MCP - Henry Habib

Table of Contents

1. Resources . . . 2
2. Context Switching . . . 2
3. Section 1 . . . 2
   3.1. Why does MCP exists? . . . 2
   3.2. Course Roadmap . . . 3
4. Section 2 . . . 6
5. Section 3 . . . 6
   5.1. Environment Setup . . . 6
6. Section 4 . . . 6
   6.1. 21. Create a Simple MCP Server1 . . . 6
   6.2. 22. Create a Simple MCP Server2 . . . 7
   6.3. Integration with Claude . . . 7
   6.4. Integration with VS Vode . . . 8
7. Section 1 . . . 14
   7.1. Why does MCP exists? . . . 14
   7.2. Course Roadmap . . . 16
8. Section 2 . . . 18
9. Section 3 . . . 18
   9.1. Environment Setup . . . 18
10. Section 4 . . . 18
   10.1. 21. Create a Simple MCP Server1 . . . 19
   10.2. 22. Create a Simple MCP Server2 . . . 20
   10.3. 23). Connect an MCP Client to an MCP Server locally built . . . 28
   10.4. 24). Connect an MCP Client to an MCP Server using NPX . . . 28
11. Section 5 . . . 29
   11.1. 25). MCP Server and local files 1 . . . 29
   11.2. 26). MCP Server and local files 2 . . . 29
   11.3. 27). Using MCP Servers to interact with your desktop . . . 29
   11.4. 28). Using MCP Servers to make API calls . . . 29
   11.5. 29. Using MCP Servers with other AI models or web search . . . 29
12. Section 5: MCP Server Deep Dive - Resources and Prompts . . . 29
   12.1. 31). Introduction to MCP Resources and Prompts . . . 29
   12.2. 32). Prompts deep dive . . . 29
   12.3. 33). Resources deep dive . . . 30
13. Section 9: MCp Client Deep Dive . . . 30
   13.1. 44). Introduction to MCP Clients . . . 30

14. Glossary 31
14.1. What is `continue` 31

1. Resources

- https://modelcontextprotocol.io/specification/2025-03-26/server/prompts
- The code repository is available in Section 3, Video 19.

2. Context Switching

- Created MCP server, and called it using Inspector.
- Create MCP server, and called using Claude. Manually update config file or use MCP CLI.
- Created MCP server, and called using Visual Studio Code.

3. Section 1

3.1. Why does MCP exists?

- Why **Model Context Protocol (MCP)** exists
  - MCP addresses limitations of Large Language Models (LLMs) when interacting with external systems
- Limitation of LLMs
  - LLMs are strong at generating content and providing conversational interfaces
    - Examples include writing emails, drafting marketing campaigns, and generating text
  - LLMs cannot directly perform real-world actions
    - Sending emails
    - Running advertisements
    - Interacting with external services
- Tool and Function Calling as a Solution
  - Tool calling allows LLMs and agents to interact with external systems
  - An **agent** can be defined as an LLM combined with a set of tools
- Lack of Standardization in Tool Calling
  - Different frameworks implement tool calling differently
    - OpenAI Agents Framework uses strict Python function-based tools
    - LangChain uses a different mechanism
  - No universal standard exists for how LLMs call external tools
- Duplicate Development of Tool Integrations

    - Developers repeatedly build the same integrations independently
        - Example: many developers build their own interface to the Gmail API
    - This leads to duplicated effort across the industry
- Maintenance Problems with Custom Integrations
    - External APIs change over time
        - Example: Gmail API updates can break existing integrations
    - Developers must constantly maintain and update their custom tool interfaces
- Limitation of Server-Based Tool Calling
    - Most existing tool calls rely on remote APIs and servers
        - Examples include Gmail API, Weather APIs, Airbnb APIs, and web services
    - This prevents LLMs from easily interacting with a user's local environment
- Opportunities for Local Machine Integration
    - Many useful AI interactions involve local machine resources
        - Organizing local files
        - Monitoring screen activity to suggest productivity improvements
        - Reading local source code in real time in editors like VS Code
- Core Purpose of MCP
    - Provide a standardized way for AI systems to interact with external tools and systems
    - Enable interoperability between different AI models and external services
- Key Benefit of MCP Standardization
    - Developers can create MCP servers that expose tools and resources
    - Any MCP client can connect to any MCP server
    - This creates a reusable and interoperable ecosystem for AI integrations
- Nature of MCP
    - MCP is not a revolutionary technology
    - It is a widely adopted **standard protocol** that simplifies AI integrations

---

3.2. Course Roadmap

- **Course Roadmap Overview**
    - The course is structured in modules to gradually build expertise in MCP
    - Students can navigate modules in any order, but some sections are strongly recommended
- **Introduction Module**
    - Provides basic orientation about the course

        - Course roadmap
        - Instructor introduction
        - How to succeed in the course
        - How to access course resources
- **MCP Architecture Overview**
    - Explains the architecture behind MCP
    - Does not require coding knowledge
    - Serves as a foundational module for the rest of the course
        - Helps all learners understand how MCP works internally
        - Skipping this section can make later modules difficult
- **Environment Setup**
    - Guides learners to prepare their development environment
    - Required for following the hands-on exercises
        - Students are encouraged to implement examples on their own machines
        - Practical learning is emphasized over passive watching
- **Core Practical Modules**
    - The main part of the course contains multiple modules focused on building MCP solutions
- **MCP Quick Start**
    - Introduces MCP through simple hands-on examples
        - Create basic MCP servers and clients
        - Build simple "Hello World" implementations
        - Learn how to connect MCP servers with MCP hosts
- **MCP Server Deep Dive**
    - Detailed exploration of how MCP servers work
    - Tools
        - Main functionality of most MCP servers
        - Around 95% of MCP servers mainly expose tools
    - Other MCP Primitives
        - Resources
        - Prompts
    - Additional Server Concepts
        - Debugging and logging
        - Local APIs vs remote APIs
        - FastMCP vs standard server setup
        - Deployment and publishing of MCP servers

    - Transport Mechanisms
        - Standard Input/Output
        - Streamable connections
        - Server-Sent Events (SSE)
- **MCP Client Deep Dive**
    - Focuses on building MCP clients
    - Key Topics
        - Client protocol
        - Connecting to MCP servers
        - Calling tools from servers
    - Optional Module
        - Can be skipped if the goal is only to build MCP servers
        - Required for building AI agents or applications that connect to MCP servers
- **MCP Integrations**
    - Demonstrates how MCP servers integrate with different platforms
    - Examples
        - AI agent frameworks
            - OpenAI Agent SDK
            - LangGraph
        - No-code or low-code tools
            - n8n
            - Copilot Studio
- **Building Real MCP Projects**
    - Practical implementation section
        - Build multiple MCP servers and clients from scratch
        - Understand the complete MCP lifecycle
        - Apply concepts learned in previous modules
- **Course Conclusion**
    - Final summary of the course
    - Certificate awarded after completion

---

4. Section 2

---

5. Section 3

5.1. Environment Setup

- VSCode
- Python
- Claude Desktop
- uv
- npm
- OpenAI API Key
- GitHub account
- (Optional) Microsoft Azure account / Virtual Machine

---

6. Section 4

```
"mcpServers": {
    "playwright": {
        "commands": "npx",
        "args": [ "@playwright/mcp@latest" ]
    }
}
```

- The Playwright MCP server will run locally on your machine because the command `npx @playwright/mcp@latest` is executed on your system. Downloads playwright/mcp and runs the Playwright MCP server
- Claude acts as the MCP client.

6.1. 21. Create a Simple MCP Server1

IMPORTANT	Create a simple MCP Server and run it locally. Open inspector with token pre-filled: http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=5976d1038450b3d335654d5b185eef35898597c65

648a1a580a76f23a6cfa1ed

`uv init`

- Initialize a new Python project using uv
- This creates basic project files like pyproject.toml

`uv venv`

- Create a virtual environment in the project directory
- By default it creates a folder named .venv

`source .venv/bin/activate`

- Activate the virtual environment
- After activation, Python and installed packages will be used from .venv

---

`uv add "mcp[cli]"`

- Install the MCP package with CLI support
- "mcp[cli]" means install MCP along with optional CLI dependencies

---

`mcp dev weather.py`

- Start the MCP development server using the file weather.py
- This runs the MCP server locally so you can test tools/resources
- NC: Create Host and Client for us

`mcp install`

6.2. 22. Create a Simple MCP Server2

- ☐ Inspector
- ☐ Claude: We want claude to run `mcp dev weather.py`

6.3. Integration with Claude

- Code

```
{
"weather": {
"command": "uv",
"args": [
    "run",
    "--directory",
    "/Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-
Code/hello-world",
    "mcp",
    "run",
    "weather.py"
]
}
```

- Prompt

`Use the weather MCP server to get the weather for London.`

6.4. Integration with VS Vode

- Visual Studio Code
- Continue Extension
- Continue CLI
- OpenAI models
- MCP (Model Context Protocol) server
- Python MCP server executed using `uv`

This setup allows an AI model to call external tools (like a weather API) through MCP.

---

1. Architecture Overview

The overall architecture works as follows:

```
VS Code / Terminal
▯
Continue Extension / Continue CLI
▯
OpenAI Model (GPT)
▯
MCP Protocol
```

```
▯
MCP Server (weather.py)
▯
External Tool / API
```

In this setup:

- The LLM decides when to call a tool.
- MCP provides the standard interface.
- The MCP server executes the tool.

---

2. Install Required Tools

Install the following tools.

- **Install Continue CLI**: `npm install -g @continuedev/cli`
- **Verify installation:**: `cn --help`
- **Install VC Code Continue Extension** After installing, restart VS Code.

---

3. Continue Configuration Directory

ls ~/.continue

```
~/.continue
config.ts
config.yaml
tsconfig.json
package.json
```

---

4. MCP Server

The MCP server in this setup is a Python program:

weather.py

It is executed using:

`uv run --directory <project-dir> mcp run weather.py`

Example project path:

`/Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-Code/hello-world`

---

5. Continue Configuration for VS Code

VS Code uses the TypeScript configuration file:

`~/.continue/config.ts`

Example configuration:

```
const config = {
defaultModel: "GPT-4o-mini",

models: [
{
title: "GPT-4o-mini",
provider: "openai",
model: "gpt-4o-mini",
apiKey: process.env.OPENAI_API_KEY
}
],

mcpServers: {
weather: {
command: "uv",
args: [
"run",
"--directory",
"/Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-
Code/hello-world",
"mcp",
"run",
"weather.py"
]
}
}
};

export default config;
```

Notes:

- `defaultModel` defines which model Continue will use.
- `mcpServers` registers available MCP tools.

---

6. Continue CLI Configuration

The Continue CLI uses a YAML configuration file:

~/.continue/config.yaml

Example configuration:

```
name: Local Config
version: 1.0.0

models:

* name: GPT-5.2
  provider: openai
  model: gpt-5.2
  apiKey: YOUR_OPENAI_KEY

mcpServers:

* name: weather
  command: uv
  args:

  * run
  * --directory
  * /Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-
Code/hello-world
  * mcp
  * run
  * weather.py
```

7. Environment Variable for API Key

Instead of hardcoding API keys, use environment variables.

Set the OpenAI key:

---

export OPENAI_API_KEY="your-api-key"

Make it permanent:

```
echo 'export OPENAI_API_KEY="your-api-key"' >> ~/.zshrc
```

```
source ~/.zshrc
```

---

8. Restart VS Code

After updating configuration: Quit VS Code completely. Restart VS Code.

---

9. Test in VS Code

Open Continue chat using Cmd + L

Example prompt:

Use the weather MCP server to get the weather in Tokyo

The system will:

1. Detect tool availability
2. Call `get_weather`
3. Return the result.

---

10. Test Using Continue CLI

Run: `cn chat --config ~/.continue/config.yaml`

Then enter prompt:

`Use the weather MCP server to get the weather in Tokyo`

The CLI will start the MCP server and call the tool.

---

11. Verify MCP Server Logs

Typical logs look like:

Processing request: ListToolsRequest Message from client Message from server

These confirm that the MCP protocol is working.

---

12. Useful Commands

Check Continue CLI:

---

cn --help

Start chat:

---

cn chat

Run a single prompt:

---

cn run "Get weather in Tokyo"

---

13. Troubleshooting

Model not visible in VS Code

Check:

- `config.ts` exists
- Continue cloud is disabled
- VS Code restarted

Cannot find process.env

Install Node types:

Install Node types:

```
cd ~/.continue
npm install --save-dev @types/node
```

Update `tsconfig.json`:

```
"types": ["node"]
```

MCP server not starting

Test manually:

```
uv run --directory <project-dir> mcp run weather.py
```

If it works manually, Continue should also run it.

---

14. Future MCP Extensions

Additional useful MCP servers:

- Filesystem MCP (read codebase)
- GitHub MCP (repo access)
- SQL MCP (database queries)
- REST API MCP

These allow AI to interact with development tools and infrastructure.

---

15. Summary

This setup enables:

- AI-assisted development in VS Code
- AI tool execution via MCP
- Terminal-based AI workflows via Continue CLI
- Custom Python tools callable by LLMs

This forms a complete local MCP development environment.

7. Section 1

7.1. Why does MCP exists?

- Why **Model Context Protocol (MCP)** exists
  - MCP addresses limitations of Large Language Models (LLMs) when interacting with external systems
- Limitation of LLMs
  - LLMs are strong at generating content and providing conversational interfaces
    - Examples include writing emails, drafting marketing campaigns, and generating text
  - LLMs cannot directly perform real-world actions
    - Sending emails

        - Running advertisements
        - Interacting with external services
- Tool and Function Calling as a Solution
    - Tool calling allows LLMs and agents to interact with external systems
    - An **agent** can be defined as an LLM combined with a set of tools
- Lack of Standardization in Tool Calling
    - Different frameworks implement tool calling differently
        - OpenAI Agents Framework uses strict Python function-based tools
        - LangChain uses a different mechanism
    - No universal standard exists for how LLMs call external tools
- Duplicate Development of Tool Integrations
    - Developers repeatedly build the same integrations independently
        - Example: many developers build their own interface to the Gmail API
    - This leads to duplicated effort across the industry
- Maintenance Problems with Custom Integrations
    - External APIs change over time
        - Example: Gmail API updates can break existing integrations
    - Developers must constantly maintain and update their custom tool interfaces
- Limitation of Server-Based Tool Calling
    - Most existing tool calls rely on remote APIs and servers
        - Examples include Gmail API, Weather APIs, Airbnb APIs, and web services
    - This prevents LLMs from easily interacting with a user's local environment
- Opportunities for Local Machine Integration
    - Many useful AI interactions involve local machine resources
        - Organizing local files
        - Monitoring screen activity to suggest productivity improvements
        - Reading local source code in real time in editors like VS Code
- Core Purpose of MCP
    - Provide a standardized way for AI systems to interact with external tools and systems
    - Enable interoperability between different AI models and external services
- Key Benefit of MCP Standardization
    - Developers can create MCP servers that expose tools and resources
    - Any MCP client can connect to any MCP server
    - This creates a reusable and interoperable ecosystem for AI integrations
- Nature of MCP

    - MCP is not a revolutionary technology
    - It is a widely adopted **standard protocol** that simplifies AI integrations

---

7.2. Course Roadmap

- **Course Roadmap Overview**
    - The course is structured in modules to gradually build expertise in MCP
    - Students can navigate modules in any order, but some sections are strongly recommended
- **Introduction Module**
    - Provides basic orientation about the course
        - Course roadmap
        - Instructor introduction
        - How to succeed in the course
        - How to access course resources
- **MCP Architecture Overview**
    - Explains the architecture behind MCP
    - Does not require coding knowledge
    - Serves as a foundational module for the rest of the course
        - Helps all learners understand how MCP works internally
        - Skipping this section can make later modules difficult
- **Environment Setup**
    - Guides learners to prepare their development environment
    - Required for following the hands-on exercises
        - Students are encouraged to implement examples on their own machines
        - Practical learning is emphasized over passive watching
- **Core Practical Modules**
    - The main part of the course contains multiple modules focused on building MCP solutions
- **MCP Quick Start**
    - Introduces MCP through simple hands-on examples
        - Create basic MCP servers and clients
        - Build simple "Hello World" implementations
        - Learn how to connect MCP servers with MCP hosts
- **MCP Server Deep Dive**
    - Detailed exploration of how MCP servers work

    - Tools
        - Main functionality of most MCP servers
        - Around 95% of MCP servers mainly expose tools
    - Other MCP Primitives
        - Resources
        - Prompts
    - Additional Server Concepts
        - Debugging and logging
        - Local APIs vs remote APIs
        - FastMCP vs standard server setup
        - Deployment and publishing of MCP servers
    - Transport Mechanisms
        - Standard Input/Output
        - Streamable connections
        - Server-Sent Events (SSE)
- **MCP Client Deep Dive**
    - Focuses on building MCP clients
    - Key Topics
        - Client protocol
        - Connecting to MCP servers
        - Calling tools from servers
    - Optional Module
        - Can be skipped if the goal is only to build MCP servers
        - Required for building AI agents or applications that connect to MCP servers
- **MCP Integrations**
    - Demonstrates how MCP servers integrate with different platforms
    - Examples
        - AI agent frameworks
            - OpenAI Agent SDK
            - LangGraph
        - No-code or low-code tools
            - n8n
            - Copilot Studio
- **Building Real MCP Projects**
    - Practical implementation section

        - Build multiple MCP servers and clients from scratch
        - Understand the complete MCP lifecycle
        - Apply concepts learned in previous modules
- **Course Conclusion**
    - Final summary of the course
    - Certificate awarded after completion

---

8. Section 2

---

9. Section 3

9.1. Environment Setup

- VSCode
- Python
- Claude Desktop
- uv
- npm
- OpenAI API Key
- GitHub account
- (Optional) Microsoft Azure account / Virtual Machine

---

10. Section 4

```
"mcpServers": {
    "playwright": {
        "commands": "npx",
        "args": [ "@playwright/mcp@latest" ]
    }
}
```

- The Playwright MCP server will run locally on your machine because the command `npx @playwright/mcp@latest` is executed on your system. Downloads playwright/mcp and runs the

Playwright MCP server

- Claude acts as the MCP client.

10.1. 21. Create a Simple MCP Server1

IMPORTANT	Create a simple MCP Server and run it locally. Open inspector with token pre-filled: http://localhost:6274/?MCP_PROXY_AUTH_TOKEN=5976d1038450b3d335654d5b185eef35898597c65648a1a580a76f23a6cfa1ed

`uv init`

- Initialize a new Python project using uv
- This creates basic project files like pyproject.toml

`uv venv`

- Create a virtual environment in the project directory
- By default it creates a folder named .venv

`source .venv/bin/activate`

- Activate the virtual environment
- After activation, Python and installed packages will be used from .venv

---

`uv add "mcp[cli]"`

- Install the MCP package with CLI support
- "mcp[cli]" means install MCP along with optional CLI dependencies

---

`mcp dev weather.py`

- Start the MCP development server using the file weather.py
- This runs the MCP server locally so you can test tools/resources
- NC: Create Host and Client for us

`uv add requests`: Install a package called requests

`mcp install weather.py`: It will install in claude. Pending

10.2. 22. Create a Simple MCP Server2

- ☐ Inspector
- ☐ Claude: We want claude to run `mcp dev weather.py`

Integration with Claude

- Code

```
{
"weather": {
"command": "uv",
"args": [
    "run",
    "--directory",
    "/Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-Code/hello-world",
    "mcp",
    "run",
    "weather.py"
]
}
```

- Prompt

`Use the weather MCP server to get the weather for London.`

Integration with VS Vode

- Visual Studio Code
- Continue Extension
- Continue CLI
- OpenAI models
- MCP (Model Context Protocol) server
- Python MCP server executed using `uv`

This setup allows an AI model to call external tools (like a weather API) through MCP.

---

1. Architecture Overview

The overall architecture works as follows:

```
VS Code / Terminal
        ▯
Continue Extension / Continue CLI
        ▯
OpenAI Model (GPT)
        ▯
MCP Protocol
        ▯
MCP Server (weather.py)
        ▯
External Tool / API
```

In this setup:

- The LLM decides when to call a tool.
- MCP provides the standard interface.
- The MCP server executes the tool.

---

2. Install Required Tools

Install Continue CLI: `npm install -g @continuedev/cli`

Verify installation:: `cn --help`

Install VS Code Continue Extension: After installing, restart VS Code.

---

3. Continue Configuration Directory

`ls ~/.continue`

```
~/.continue
 ├─── config.ts
 ├─── config.yaml
 ├─── tsconfig.json
 └─── package.json
```

---

4). MCP Server

`mcp install weather.py`: It will install in claude. Pending

Both commands start your **MCP server**, but in **different modes**.

- **Run MCP server (normal mode)**

```
uv run --directory /Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-Code/hello-world mcp run weather.py
```

What it does:

- Starts the **MCP server**
- Waits for a **client** to connect (for example **Continue** or **Claude Desktop**)
- No UI
- No debugging interface

Architecture:

```
Continue / Claude
▯
MCP Server (weather.py)
```

Terminal usually shows **no output** because it is waiting for requests.

- **Run MCP server with inspector (development mode)**

```
mcp dev weather.py
```

What it does:

- Starts the **same MCP server**
- Also starts a **debug UI (Inspector)**

Architecture:

```
Browser Inspector
▯
MCP Proxy
▯
```

```
weather.py (MCP Server)
```

You get:

- Tool testing UI
- Request / response logs
- Debugging

- **Simple way to remember**

Command	Purpose
`mcp run weather.py`	Run server for real clients
`mcp dev weather.py`	Run server + debugging inspector

- **Typical development workflow**

Step 1 — Build tool Step 2 — Test with inspector

```
mcp dev weather.py
```

Step 3 — Integrate with AI client

- **Visual Studio Code** + Continue
- **Claude Desktop**

Those clients automatically run:

```
mcp run weather.py
```

- **Important**

When using **Continue in VS Code**, you **do NOT manually run**:

```
mcp run weather.py
```

because Continue will start the MCP server automatically from your config.

---

5. Continue Configuration for VS Code

VS Code uses the TypeScript configuration file:

`~/.continue/config.ts`

Example configuration:

```
const config = {
defaultModel: "GPT-4o-mini",

models: [
{
title: "GPT-4o-mini",
provider: "openai",
model: "gpt-4o-mini",
apiKey: process.env.OPENAI_API_KEY
}
],

mcpServers: {
weather: {
command: "uv",
args: [
"run",
"--directory",
"/Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-
Code/hello-world",
"mcp",
"run",
"weather.py"
]
}
}
};

export default config;
```

Notes:

- `defaultModel` defines which model Continue will use.
- `mcpServers` registers available MCP tools.

---

6. Continue CLI Configuration

The Continue CLI uses a YAML configuration file:

~/.continue/config.yaml

Example configuration:

```
name: Local Config
version: 1.0.0

models:

* name: GPT-5.2
  provider: openai
  model: gpt-5.2
  apiKey: YOUR_OPENAI_KEY

mcpServers:

* name: weather
  command: uv
  args:

  * run
  * --directory
  * /Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-
Code/hello-world
  * mcp
  * run
  * weather.py
```

---

7. Environment Variable for API Key

- export OPENAI_API_KEY="your-api-key"

---

9. Test in VS Code

Open Continue chat (Cmd + L)

Example prompt:

Use the weather MCP server to get the weather in Tokyo

The system will:

1. Detect tool availability
2. Call `get_weather`
3. Return the result.

---

10. Test Using Continue CLI

Run:

---

cn chat --config ~/.continue/config.yaml

Then enter:

Use the weather MCP server to get the weather in Tokyo

The CLI will start the MCP server and call the tool.

---

11. Verify MCP Server Logs

Typical logs look like:

Processing request: ListToolsRequest Message from client Message from server

These confirm that the MCP protocol is working.

---

12. Useful Commands

Check Continue CLI:

---

cn --help

Start chat:

---

cn chat

Run a single prompt:

---

cn run "Get weather in Tokyo"

---

13. Troubleshooting

Model not visible in VS Code

Check:

- `config.ts` exists
- Continue cloud is disabled
- VS Code restarted

Cannot find process.env

Install Node types:

```
cd ~/.continue
npm install --save-dev @types/node
```

Update `tsconfig.json`:

```
"types": ["node"]
```

MCP server not starting

Test manually:

`uv run --directory <project-dir> mcp run weather.py`

If it works manually, Continue should also run it.

---

14. Future MCP Extensions

Additional useful MCP servers:

- Filesystem MCP (read codebase)
- GitHub MCP (repo access)
- SQL MCP (database queries)
- REST API MCP

These allow AI to interact with development tools and infrastructure.

---

15. Summary

This setup enables:

- AI-assisted development in VS Code
- AI tool execution via MCP
- Terminal-based AI workflows via Continue CLI
- Custom Python tools callable by LLMs

This forms a complete local MCP development environment.

10.3. 23). Connect an MCP Client to an MCP Server locally built

- I am inside client folder, and doing all this.

```
uv init

uv venv

source ./venv/bin/activate

uv add "mcp[cli]"

uv run client_weather.py

uv run client_airbnb.py
```

- Calling Weather with my own client

10.4. 24). Connect an MCP Client to an MCP Server using NPX

- Calling Airbnb with my own client

---

11. Section 5

11.1. 25). MCP Server and local files 1

- /Users/nareshchaurasia/nc/Python-TechHub/AI-ML-GenAI/MCP/MCP-Henry-Habib/My-Code/notes
- `mcp dev local.py`
    - Open the inspector and perform read and write operations.

11.2. 26). MCP Server and local files 2

- Adding it to claude and running from Claude

11.3. 27). Using MCP Servers to interact with your desktop

11.4. 28). Using MCP Servers to make API calls

- Refer code. I just tried it with Inspector.

11.5. 29. Using MCP Servers with other AI models or web search

12. Section 5: MCP Server Deep Dive - Resources and Prompts

12.1. 31). Introduction to MCP Resources and Prompts

12.2. 32). Prompts deep dive

- It works with Inspector and Not with Claude.

12.3. 33). Resources deep dive

- It works with Inspector and Not with Claude.

---

13. Section 9: MCp Client Deep Dive

13.1. 44). Introduction to MCP Clients

▼ *MCP Architecture*

[MCP Architecture Diagram] | *mcp/mcp-arch.png*

- Using Visual Studio Code
  - You typically do **not** need to learn MCP clients
    - MCP client functionality is already built-in
    - Connection to MCP servers is handled automatically
- What happens behind the scenes
  - VS Code acts as an MCP host
    - Includes an MCP client
      - Uses MCP protocol internally
      - Connects to MCP servers for you
- When you can safely skip this topic
  - Using existing tools or IDEs
    - VS Code, Cursor, etc.
  - Using frameworks
    - LangGraph
    - OpenAI Agents SDK
      - They abstract MCP completely
- When you would need it
  - Building your own tool or app from scratch
    - Creating your own MCP host
    - Manually handling client-server communication
- Bottom line
  - Normal development
    - Skip it

- Infrastructure-level development
  - Learn it

14. Glossary

- Cline VS Code Plugin

14.1. What is `continue`

▼ *Continue Plugin*

Continue is an AI coding assistant for developers that runs inside editors like Visual Studio Code and in the terminal.

- It connects **LLMs (like GPT, Claude, etc.)** with your **codebase and tools**.
- It lets you **chat with AI about your code**, generate code, refactor, or debug.
- It supports **tool calling using MCP (Model Context Protocol)** so AI can execute external tools (APIs, databases, scripts).
- It works with multiple models (OpenAI, Anthropic, local models, etc.).
- It also provides a **CLI (`cn`)** to run AI workflows from the terminal.

In simple terms:

Continue = AI layer that connects your editor + LLM + tools.

Typical flow:

```
Editor / Terminal
      ▯
Continue
```

```
      ▯
LLM (GPT / Claude)
      ▯
Tools via MCP
      ▯
Your code / APIs / databases
```

So Continue acts like a **bridge between your development environment and AI models**.